
COMPLEX ENGINEERING PROBLEM

Git Branching and Merge-Conflict Resolution

PREET HANSALIYA [92510103060]
INDRAPRIT JADAV[92510103055]
OM BERA [92510103074]

DEPARTMENT
COMPUTER ENGINEERING

ACADEMIC YEAR
2026-2027

SEMESTER
5TH

SUBJECT
OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS
5EC6

BATCH
A-B

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERING PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security.	3 Marks: Good system design and technical discussion with minor omissions.	1-2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Git Branching and Merge-Conflict Resolution							

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

Git Branching and Merge-Conflict Resolution

Course Name: OPEN SOURCE TECHNOLOGIES(01CE0526)

Co's Mapped - C02

SDG's Mapped -SDG 9, SDG 17

1. Aim

To implement and demonstrate a Git-based collaborative software development workflow by creating a repository with a main branch and feature branches, making independent changes, generating a merge conflict, resolving the conflict, and verifying the final integrated version.

The activity also aims to understand how Git helps developers manage source-code changes, maintain version history, work independently on different features, and safely integrate multiple contributions into a common project.

2. Problem Statement

In collaborative software development, multiple developers often work on the same project simultaneously. When developers directly modify common files without a proper version-control workflow, changes may be overwritten or become difficult to integrate. This can result in inconsistent source-code versions, merge conflicts, loss of changes, and delays in development.

The problem is to design and implement a Git-based workflow in which a project contains a main branch and separate feature branches. Different branches must make changes to the same file so that a merge conflict can be generated intentionally. The conflict must then be identified, resolved correctly, committed, and verified.

The activity demonstrates how branching, merging, commit history, and conflict resolution can be used to support collaborative software development.

3. Problem Analysis

a) What is the engineering problem?

The engineering problem is to manage source-code changes made concurrently by multiple developers while maintaining the integrity and consistency of the project.

A suitable version-control mechanism is required to isolate individual development work, maintain a history of changes, combine completed features, and handle situations where two developers modify the same portion of a file.

Git provides the required branching and merging mechanisms for managing these changes efficiently.

b) Why is this problem complex?

The problem involves multiple engineering concepts and decision-making activities.

- ☒ Requires multiple engineering concepts
- ☒ Requires analysis and decision making
- ☒ Involves design constraints
- ☒ Requires use of engineering tools
- ☒ Requires performance, security and reliability considerations

COMPLEX ENGINEERING PROBLEM - REPORT

The problem becomes complex when multiple developers make changes to the same file at the same time. Git can automatically merge many independent changes, but when two branches modify the same lines differently, automatic merging may not be possible.

In such a situation, the developer must:

- Identify the conflicting changes.
- Understand the purpose of both changes.
- Decide which code should be retained.
- Combine changes if required.
- Remove Git conflict markers.
- Test and verify the final code.
- Commit the resolved version.

Therefore, the problem requires both technical knowledge and engineering decision-making.

4. Knowledge and Tools Required

Engineering Concepts Used:

1. Version Control

Version control is used to track modifications made to source-code files. It allows developers to maintain different versions of a project and review previous changes.

2. Git Branching and Merging

Branches allow developers to work independently without directly affecting the main project. After completing development, the changes can be merged into the main branch.

3. Merge Conflict Resolution

A merge conflict occurs when Git cannot automatically combine changes from different branches. The developer must inspect and resolve the conflicting sections manually.

4. Repository Management

A Git repository stores project files and their complete version history. GitHub can be used as a remote repository for storing and sharing the project.

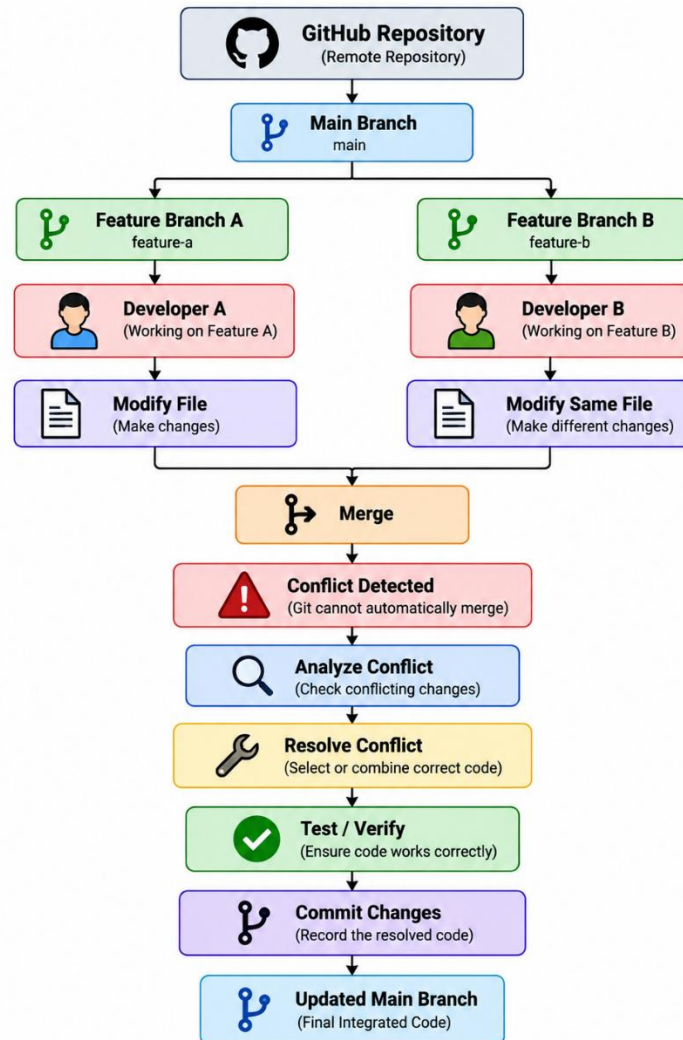
Tools/Software Used

1. **Git** – Distributed version-control system used for repository, branch, commit, and merge operations.
2. **GitHub** – Remote repository platform used for storing and managing the Git project.
3. **Visual Studio Code** – Used for editing and resolving conflicts in source files.
4. **Command Prompt / Git Bash** – Used for executing Git commands.

COMPLEX ENGINEERING PROBLEM - REPORT

5. Proposed Solution Design

Block Diagram / Architecture Diagram



[FIGURE 1: Git Branching and Merge-Conflict Resolution Workflow]

Brief Working

The proposed solution uses Git for local version control and GitHub as the remote repository. The project initially contains a main branch with the basic project files.

Two separate feature branches are created from the main branch. Each branch represents independent development work. Developers modify and commit their changes separately.

When the first feature branch is merged, its changes become part of the main branch. When the second feature branch is merged, Git compares the changes with the current main branch.

If both branches have modified different parts of a file, Git can normally merge the changes automatically. However, if both branches modify the same lines differently, Git generates a merge conflict.

The developer then examines the conflicting sections, selects or combines the required changes, removes the conflict markers, stages the resolved file, and creates a merge-resolution commit. Finally, the project is verified to ensure that the final version contains the required changes.

COMPLEX ENGINEERING PROBLEM - REPORT

6. Implementation / Procedure

Step 1: Create the Repository

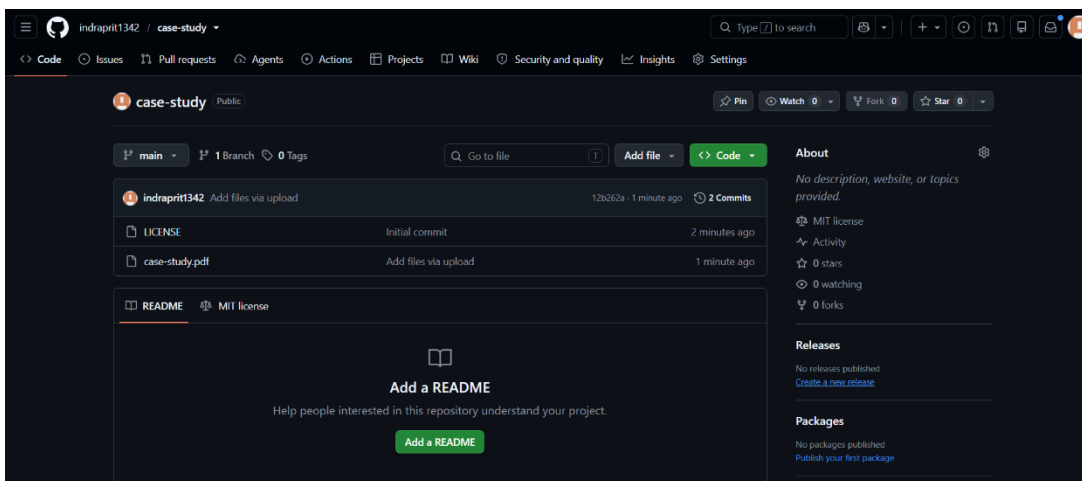
A new project repository is created using Git. The initial project files are added and committed.

git init, git add .

git commit -m "Initial commit"

git branch -M main

The repository can then be connected to GitHub and pushed to the remote repository.



[SCREENSHOT 1: GitHub Repository with Initial Commit]

Step 2: Create Feature Branches

Two feature branches are created from the main branch.

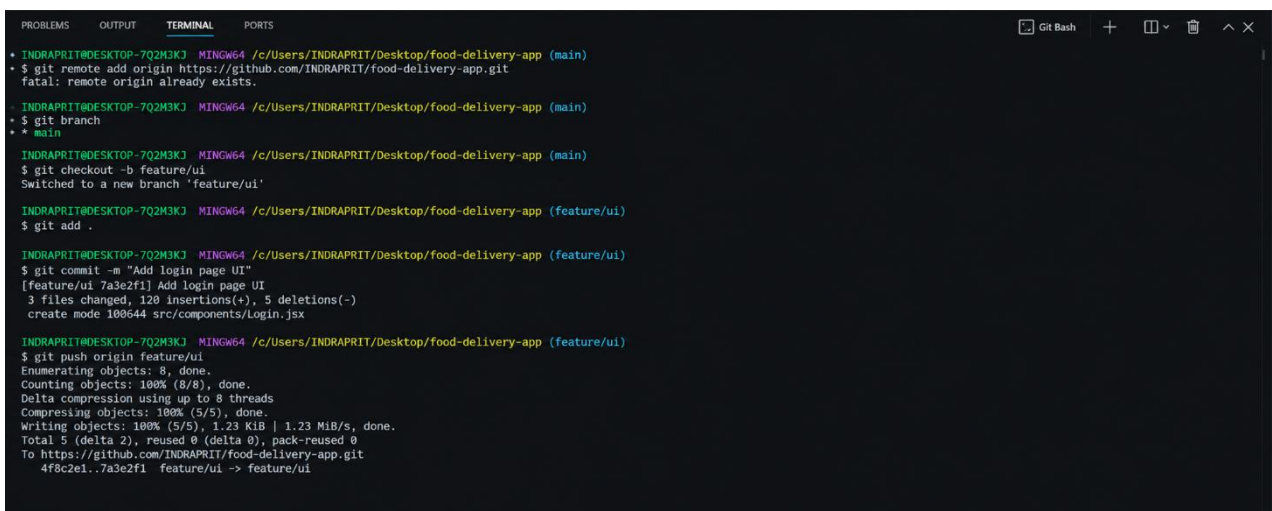
git switch -c feature-a

After completing work on the first branch, the developer switches back to main and creates another feature branch.

git switch main

git switch -c feature-b

Feature branches provide isolation between different development tasks.



[SCREENSHOT 2: Main and Feature Branches]

COMPLEX ENGINEERING PROBLEM - REPORT

Step 3: Make Changes in Feature Branch A

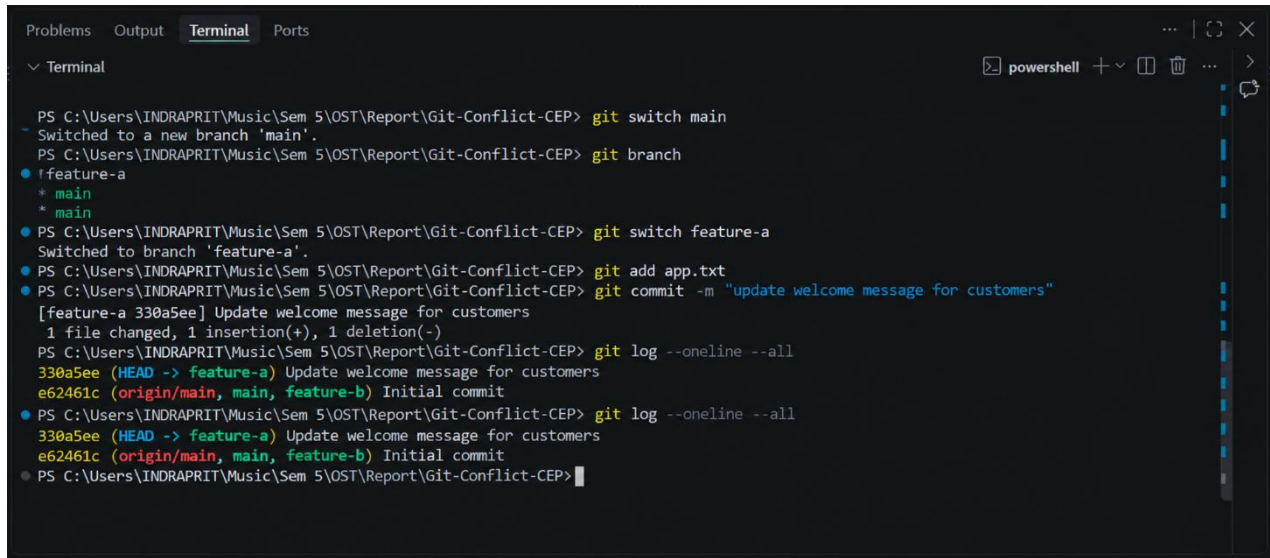
Developer A works on feature-a and modifies the selected project file.

After making the changes, the developer stages and commits the file.

git add .

git commit -m "Update feature A"

The commit creates a permanent record of the changes in the feature branch.



```
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git switch main
Switched to a new branch 'main'.
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git branch
* feature-a
* main
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git switch feature-a
Switched to branch 'feature-a'.
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git add app.txt
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git commit -m "update welcome message for customers"
[feature-a 330a5ee] Update welcome message for customers
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --all
330a5ee (HEAD -> feature-a) Update welcome message for customers
e62461c (origin/main, main, feature-b) Initial commit
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --all
330a5ee (HEAD -> feature-a) Update welcome message for customers
e62461c (origin/main, main, feature-b) Initial commit
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP>
```

[SCREENSHOT 3: Feature A Commit]

Step 4: Make Conflicting Changes in Feature Branch B

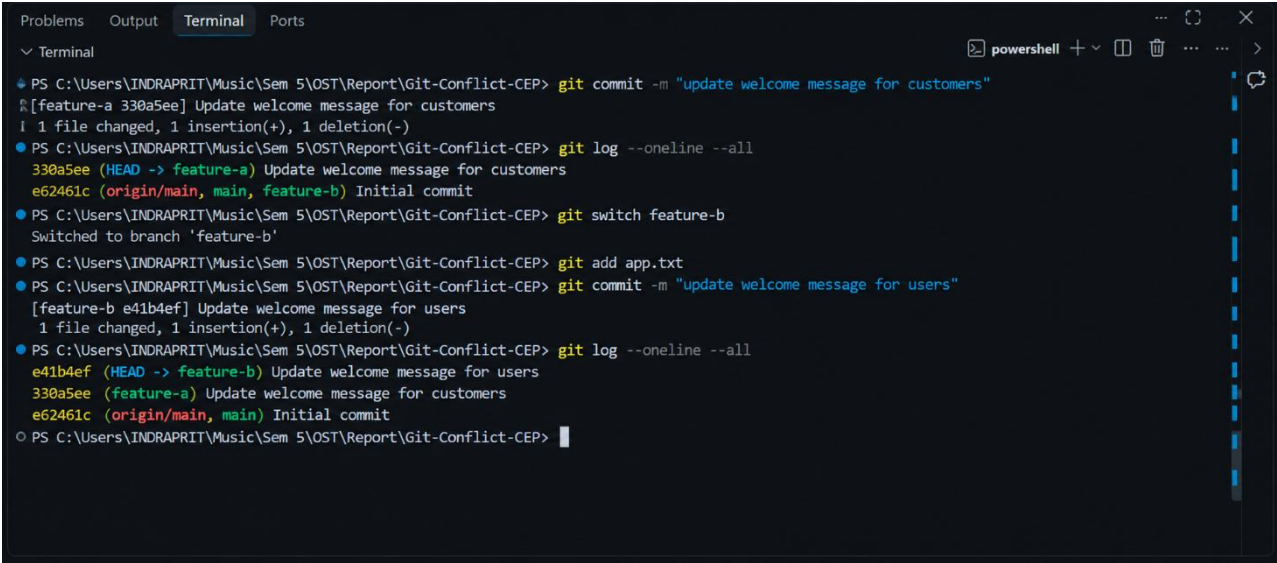
Developer B works on feature-b and modifies the same file and the same portion of the file differently. The changes are then committed.

git add .

git commit -m "Update feature B"

At this point, both feature branches contain different versions of the same portion of the file.

COMPLEX ENGINEERING PROBLEM - REPORT



```

PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git commit -m "update welcome message for customers"
[feature-a 330a5ee] Update welcome message for customers
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --all
330a5ee (HEAD -> feature-a) Update welcome message for customers
e62461c (origin/main, main, feature-b) Initial commit
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git switch feature-b
Switched to branch 'feature-b'
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git add app.txt
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git commit -m "update welcome message for users"
[feature-b e41b4ef] Update welcome message for users
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --all
e41b4ef (HEAD -> feature-b) Update welcome message for users
330a5ee (feature-a) Update welcome message for customers
e62461c (origin/main, main) Initial commit
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP>

```

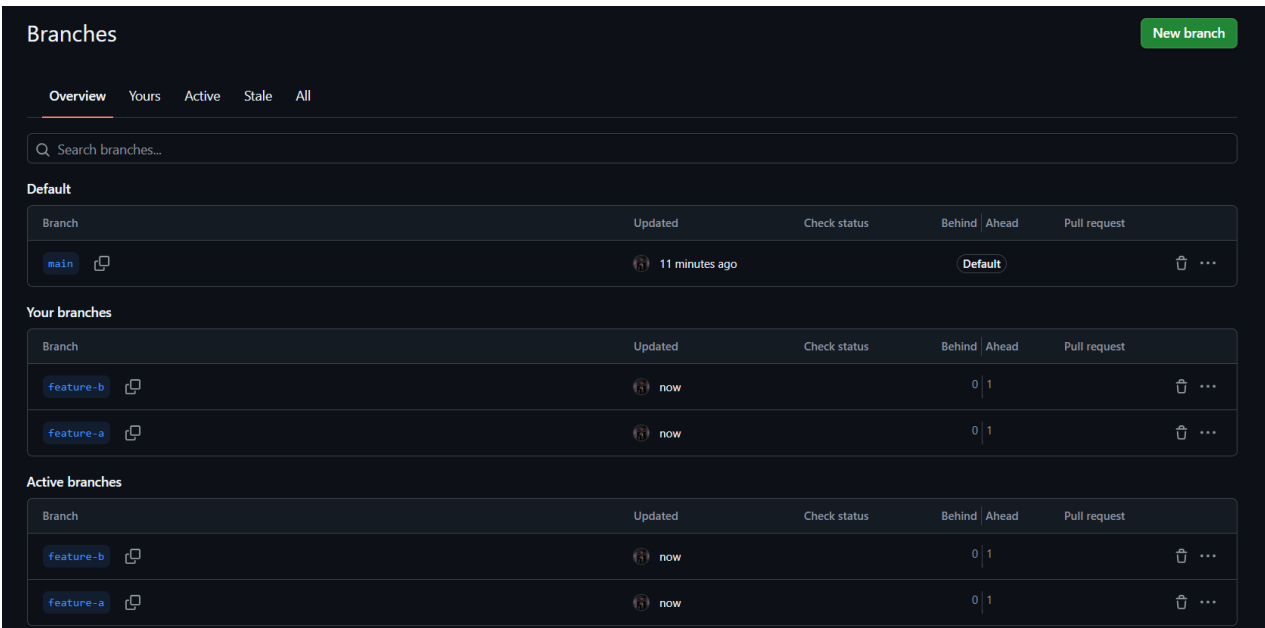
[SCREENSHOT 4: Feature B Commit]

Step 5: Push Feature Branches

The branches can be pushed to the GitHub repository.

git push -u origin feature-a, git push -u origin feature-b

The remote repository now contains the main branch and both feature branches.



Branch	Updated	Check status	Behind	Ahead	Pull request
main	11 minutes ago			Default	
feature-b	now		0	1	
feature-a	now		0	1	

[SCREENSHOT 5: Feature Branches on GitHub]

Step 6: Merge Feature Branch A

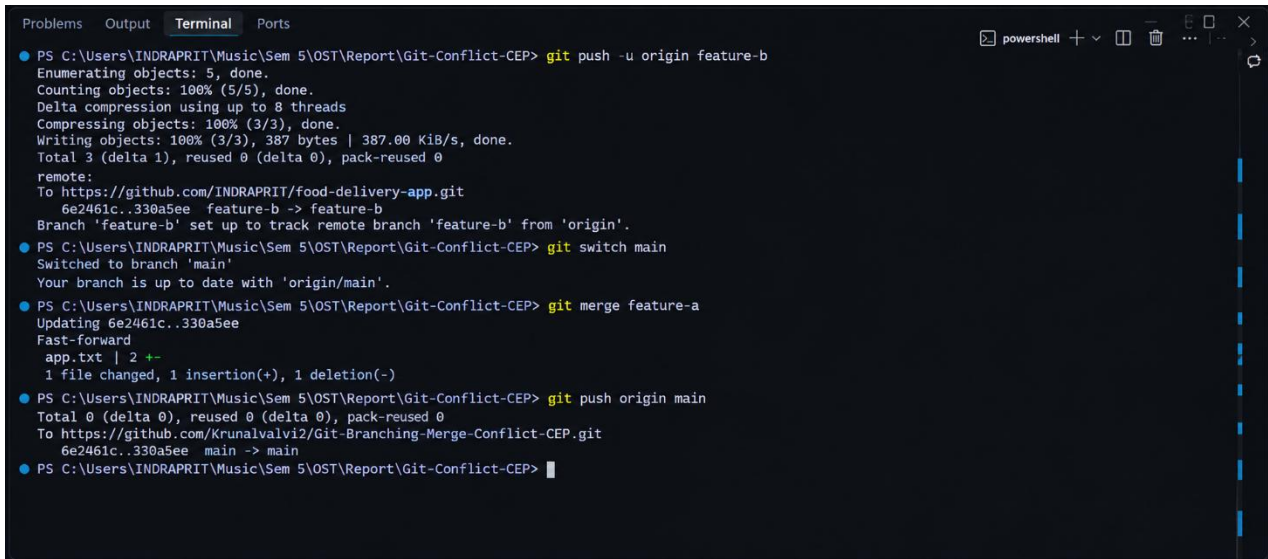
The developer switches to the main branch and merges the first feature branch.

git switch main

git merge feature-a

Since the changes from feature-a do not conflict with the current main version, the merge is completed successfully.

COMPLEX ENGINEERING PROBLEM - REPORT



```
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git push -u origin feature-b
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 387 bytes | 387.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
To https://github.com/INDRAPRIT/food-delivery-app.git
6e2461c..330a5ee feature-b -> feature-b
Branch 'feature-b' set up to track remote branch 'feature-b' from 'origin'.
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git merge feature-a
Updating 6e2461c..330a5ee
Fast-forward
 app.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP> git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Krunalvalvi2/Git-Branching-Merge-Conflict-CEP.git
6e2461c..330a5ee main -> main
PS C:\Users\INDRAPRIT\Music\Sem 5\OST\Report\Git-Conflict-CEP>
```

[SCREENSHOT 6: Successful Feature-A Merge]

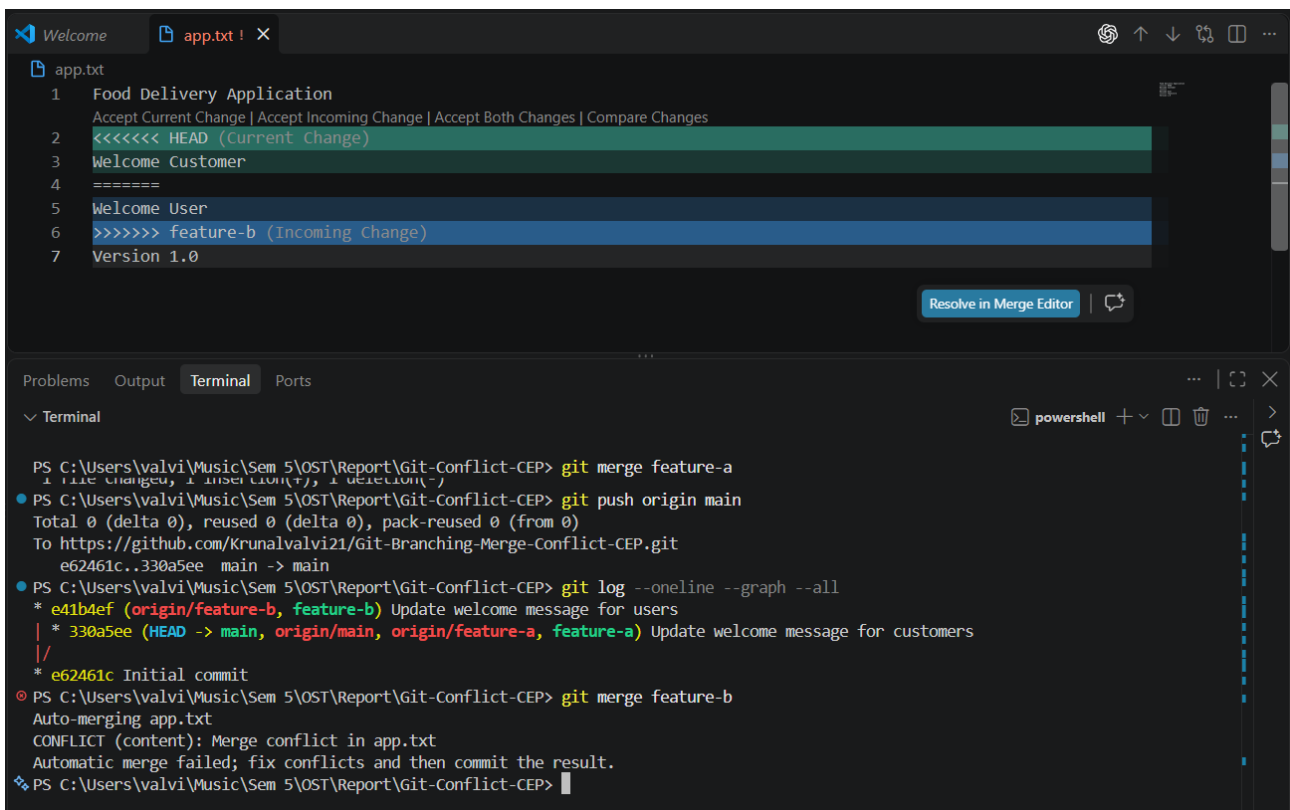
Step 7: Merge Feature Branch B

The second feature branch is now merged into the updated main branch.

git merge feature-b

Since feature-b contains a different modification to the same portion of the file, Git cannot automatically determine which change should be retained.

Git therefore reports a **merge conflict**.



```
app.txt
1 Food Delivery Application
2 Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
3 <<<<<< HEAD (Current Change)
4 Welcome Customer
5 =====
6 Welcome User
7 >>>>>> feature-b (Incoming Change)
8 Version 1.0

Resolve in Merge Editor

PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git merge feature-a
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Krunalvalvi2/Git-Branching-Merge-Conflict-CEP.git
6e2461c..330a5ee main -> main
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --graph --all
* e41b4ef (origin/feature-b, feature-b) Update welcome message for users
| * 330a5ee (HEAD -> main, origin/main, origin/feature-a, feature-a) Update welcome message for customers
|/
* e62461c Initial commit
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git merge feature-b
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP>
```

[SCREENSHOT 7: Git Merge Conflict]

COMPLEX ENGINEERING PROBLEM - REPORT

Step 8: Identify the Merge Conflict

Git adds conflict markers to the affected file.

A typical conflict appears as:

```
<<<<<< HEAD
Change from main branch
=====
Change from feature-b
>>>>>> feature-b
```

The section between <<<<<< HEAD and ===== represents the current branch version.

The section between ===== and >>>>>> feature-b represents the incoming branch version.

The developer examines both versions and determines the correct final code.

Step 9: Resolve the Conflict

The developer edits the file and keeps the required code or combines both changes when appropriate.

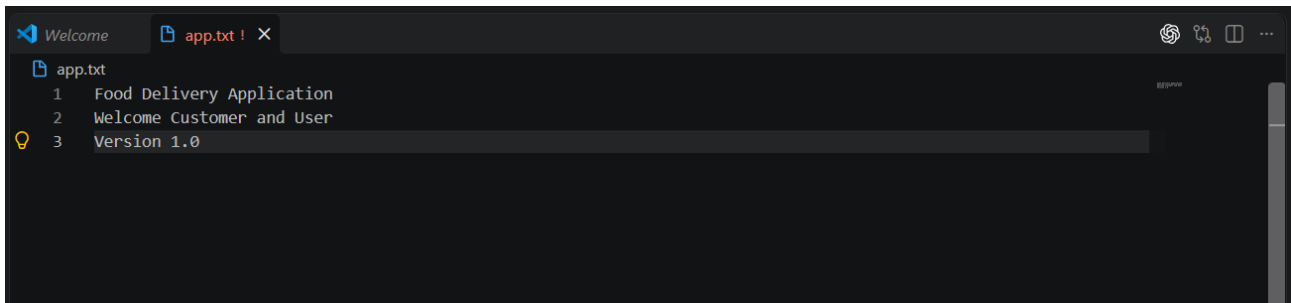
All conflict markers are removed from the final file.

After resolving the conflict, the file is staged:

```
git add <file-name>
```

The resolution is then committed:

```
git commit -m "Resolve merge conflict"
```



[SCREENSHOT 8: Resolved Conflict in Visual Studio Code]

Step 10: Verify the Final Repository

The final repository status is checked.

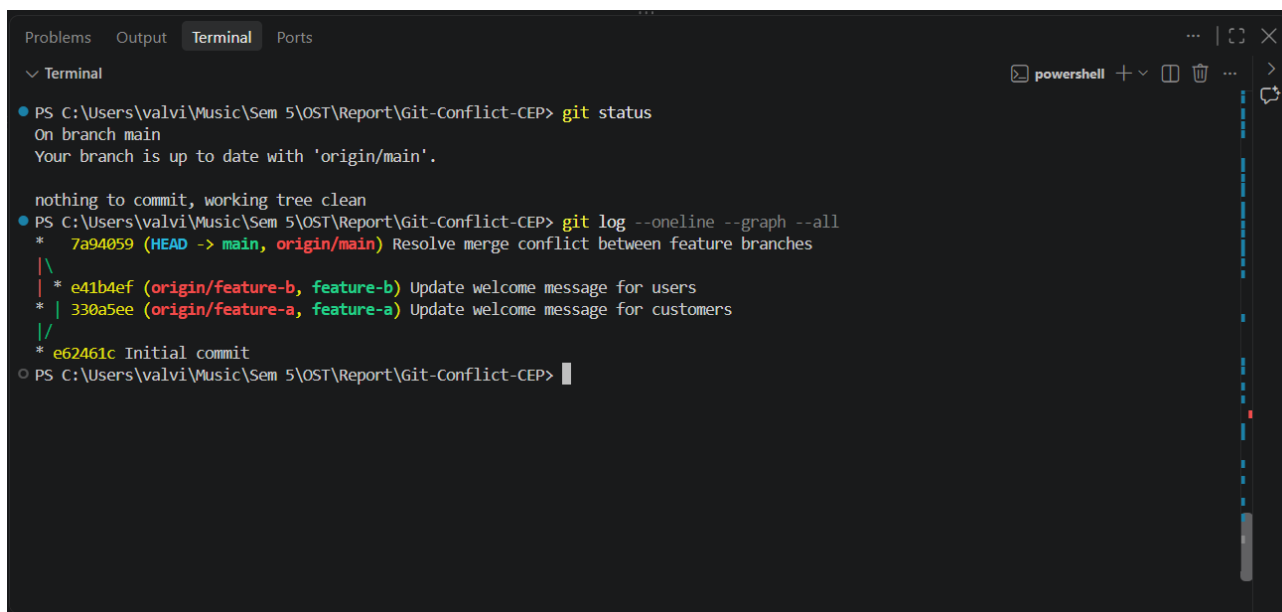
```
git status
```

The commit history can also be inspected:

```
git log --oneline --graph --all
```

The final file is checked to ensure that the required changes from the branches have been correctly integrated.

COMPLEX ENGINEERING PROBLEM - REPORT



```

Problems Output Terminal Ports
Terminal
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --graph --all
* 7a94059 (HEAD -> main, origin/main) Resolve merge conflict between feature branches
| \
| * e41b4ef (origin/feature-b, feature-b) Update welcome message for users
| * 330a5ee (origin/feature-a, feature-a) Update welcome message for customers
| /
* e62461c Initial commit
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP>
  
```

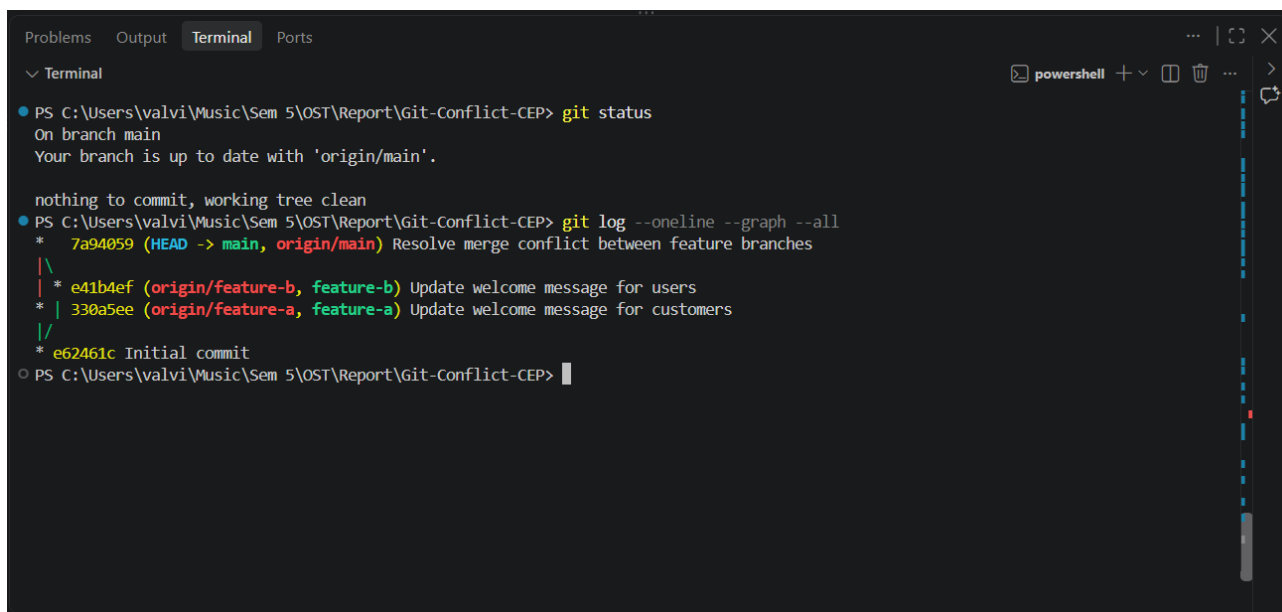
[SCREENSHOT 9: Final Git History and Repository Status]

7. Testing and Results

Testing is performed to verify the complete Git workflow, including repository creation, branch creation, commits, merging, conflict generation, conflict resolution, and final verification.

Test Parameter	Expected Result	Obtained Result
Repository creation	Git repository is created successfully	Repository was created successfully and initialized with the main branch.
Main branch	Main branch is available	main branch was created and used for final integration.
Feature branch creation	Separate feature branches are created	feature-a and feature-b were created successfully.
File modification	Changes are committed successfully	Both developers' changes were committed to their respective feature branches.
Feature branch merge	First feature branch merges successfully	feature-a was merged into main successfully.
Conflict generation	Git identifies conflicting changes	Git detected a conflict when feature-b was merged because both branches modified the same portion of the file.
Conflict resolution	Conflict is resolved successfully	The conflicting code was manually reviewed and resolved.
Final commit	Resolved changes are committed	The resolved changes were committed successfully.
Repository verification	Final repository contains integrated changes	Final repository status was clean and the commit history confirmed successful integration.

COMPLEX ENGINEERING PROBLEM - REPORT



```

Problems Output Terminal Ports
Terminal
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP> git log --oneline --graph --all
* 7a94059 (HEAD -> main, origin/main) Resolve merge conflict between feature branches
| \
| * e41b4ef (origin/feature-b, feature-b) Update welcome message for users
| * 330a5ee (origin/feature-a, feature-a) Update welcome message for customers
| /
* e62461c Initial commit
PS C:\Users\valvi\Music\Sem 5\OST\Report\Git-Conflict-CEP>
  
```

[SCREENSHOT 10: Final Testing / Git Status]

The testing demonstrates that Git can successfully isolate development activities using branches and maintain a history of changes through commits. The intentional conflicting modifications also demonstrate Git's ability to detect changes that cannot be automatically merged. After manual resolution, the final repository can be verified using Git status and commit history.

8. Challenges Faced and Solutions

Challenge	Solution
Multiple developers modify the same file	Use separate feature branches for individual development
Changes conflict during merging	Inspect the conflicting sections and manually resolve them
Accidental changes to the main branch	Perform development work using feature branches
Difficulty tracking changes	Use meaningful commit messages and Git history
Conflict markers remain in the file	Carefully inspect and remove all conflict markers
Incorrect conflict resolution	Review the final code and verify repository status
Managing remote branches	Use GitHub to store and manage branches centrally

The main challenge in the activity is understanding that Git does not automatically decide which conflicting change is technically correct. The developer must understand the purpose of the changes and select the appropriate final version.

9. Conclusion

The **Git Branching and Merge-Conflict Resolution** activity demonstrates the importance of version control in collaborative software development. Git branches allow developers to work independently without directly affecting the main project.

COMPLEX ENGINEERING PROBLEM - REPORT

The activity successfully demonstrates the creation of feature branches, modification of a common file, merging of branches, generation of a merge conflict, and manual conflict resolution. Git's commit history provides a clear record of development activities and helps track changes made by different developers. The proposed workflow reduces the risk of accidental overwriting of code and provides a structured method for integrating multiple developers' contributions. The activity also improves understanding of practical Git commands, branching strategies, merging, and conflict management.

10. SDG Mapping and Justification

SDG 9 – Industry, Innovation and Infrastructure

Git-based version control contributes to reliable software-development infrastructure by providing structured methods for managing source code and development changes.

The workflow supports software engineering practices such as version tracking, branch-based development, controlled integration, and collaborative development. These practices help development teams build and maintain software more systematically.

SDG 17 – Partnerships for the Goals

Git and GitHub support collaboration among multiple developers working on the same software project. Developers can maintain separate branches, contribute changes, review project history, and integrate their work into a common repository.

The workflow therefore supports effective technical collaboration and sharing of software development resources between team members.

11. References

- [1] Git, "Git Documentation," Git SCM. [Online]. Available: <https://git-scm.com/docs>
- [2] GitHub, "GitHub Documentation," GitHub Docs. [Online]. Available: <https://docs.github.com/>
- [3] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [4] GitHub, "About Git," GitHub Docs. [Online]. Available: <https://docs.github.com/en/get-started/learning-about-github/about-github-and-git>